

ColdStart — Proje Kurulum & Kullanım Kılavuzu

Sparse Knowledge Environments için Adaptif Hibrit RAG · Kubilay Özyalçın · Haziran 2026

Genel Bakış

ColdStart, bilgi tabanı boş başlayan RAG sistemlerindeki *cold start* problemine yönelik üç katmanlı adaptif bir arama mimarisidir. Sistem, kayıtlı belge sayısını her istekte okuyarak uygun stratejiyi çalışma anında seçer; katman geçişleri iki yönlü ve otomatiktir.

KATMAN	ARALIK	TEKNOLOJİ	LLM
Layer 1 Keyword	$n < 50$	BM25 + Türkçe-duyarlı tokenizer	Yok
Layer 2 Lightweight	$50 \leq n < 200$	OpenAI <code>text-embedding-3-small</code> + cosine similarity	Yok
Layer 3 Full RAG	$n \geq 200$	Qdrant + Semantic Kernel + GPT-4o-mini	Var

1 Önkoşullar

GEREKSİNİM	SÜRÜM / NOT	NE İÇİN
Docker Desktop	API + Qdrant container'ları	Seçenek A (önerilen) — tek başına yeterli
.NET SDK	8.0+ — <code>dotnet --version</code> ile doğrulayın	Yalnız Seçenek B
OpenAI API anahtarı	Teslim paketindeki <code>.env</code> dosyasında hazır gelir	Layer 2 ve 3
Modern tarayıcı	Demo UI için (Chrome, Safari, Edge)	—

Anahtar olmadan da proje çalışır: sistem Layer 1 (BM25) modunda tam işlevseldir. Layer 2/3 denenmek istendiğinde anahtar gerekir.

2 Kurulum — Seçenek A: Docker ile tek komut (önerilen)

.NET SDK gerektirmez; Docker Desktop yeterlidir. API anahtarı, proje kökündeki `.env` dosyasından okunur (teslim paketinde hazır gelir; yoksa `.env.example`'ı `.env` olarak kopyalayıp anahtarınızı yazın).

1 Proje klasörüne girin ve sistemi başlatın

```
cd ColdStart
docker compose up -d --build
```

İlk çalıştırmada API imajı derlenir (birkaç dakika). API + Qdrant birlikte ayağa kalkar.

2 Tarayıcıdan açın

```
http://localhost:5199
```

Swagger: `http://localhost:5199/swagger` · Durdurmak için: `docker compose down`

2b

Kurulum — Seçenek B: .NET SDK ile

1

API anahtarını doğrulayın

```
# Proje kökündeki .env dosyası otomatik okunur – paketle hazır gelir, ek adım gerekmez.  
# Dosya yoksa: .env.example'ı .env olarak kopyalayıp anahtarınızı yazın,  
# veya environment variable kullanın:  
export OPENAI_API_KEY="sk-..."          # macOS / Linux  
setx OPENAI_API_KEY "sk-..."            # Windows (yeni terminal gerekir)
```

Anahtar kod tabanında yer almaz; `.env` dosyası veya environment variable yeterlidir. Gerçek environment her zaman dosyaya göre önceliklidir.

2

Qdrant'ı başlatın (yalnız Layer 3 için)

```
docker compose up -d qdrant
```

İmaj sürümü (`qdrant/qdrant:v1.18.1`) istemci kütüphanesiyle hizalıdır — değiştirmeyin.

3

API + demo UI'yi çalıştırın

```
dotnet run --project src/ColdStart.Api
```

UI: `http://localhost:5266` · Swagger: `http://localhost:5266/swagger`

Her iki seçenekte de açılışta `data/synthetic/documents_seed.json` içindeki 15 sentetik belge otomatik yüklenir. Demo yapılandırmasında eşikler düşürülmüştür (Layer 2: 5, Layer 3: 20 belge) — katman geçişleri birkaç belge ekleyerek gözlemlenebilir.

3 Nasıl Kullanılır — Demo UI

Ekran bölümleri

- **Durum kartı** — belge sayısı, aktif katman rozeti ve bir sonraki katman eşiğine kalan belge sayısı.
- **Belge yönetimi** — metin girerek belge ekleme, tekil silme veya tümünü temizleme.
- **Arama** — sorgu kutusu; cevap kartında aktif katman rozeti, cevap metni ve skorlarıyla kaynak listesi görünür.
- **Sorgula + Değerlendir** — aynı sorguyu GPT-4o-mini hakeme puanlatır (faithfulness + answer relevancy).
- **Metrikler** — arama kayıtları ve katman geçiş zaman çizelgesi.
- **Dokümantasyon / Test sayfaları** — üst menüden mimari anlatımı ve test senaryolarına ulaşılır.

Katman geçişini gözlemlemek

1 Layer 1'i görün

— açılıştaki 15 belgeyle bir sorgu çalıştırın; cevap kartında **Layer 1** rozeti görünür.

2 Eşiği aşın

— belge sayısını 50'nin üzerine çıkarın; aynı sorgu artık **Layer 2** 'den döner (ilk istekte eksik embedding'ler toplu üretilir).

3 Full RAG'e geçin

— 200+ belgede **Layer 3** devreye girer: Qdrant'a chunk indekslenir, GPT-4o-mini kaynaklara dayalı cevap sentezler.

4 Geri dönüşü test edin

— belgeleri silince router otomatik olarak alt katmana döner; geçiş Metrikler sayfasına kaydedilir.

4 API Uç Noktaları

ENDPOINT	AÇIKLAMA
POST /api/search	Adaptif arama — aktif katman cevabı + kaynaklar
POST /api/document	Belge ekleme (10.000 karakter üst sınır)
DEL /api/document/{id} . /api/document	Tekil silme / tümünü temizleme
GET /api/status	Belge sayısı, aktif katman, sonraki eşiğe kalan
POST /api/evaluate	Arama + LLM-as-a-judge puanlama (faithfulness, relevancy)
GET /api/metrics	Arama kayıtları + katman geçiş zaman çizelgesi

Örnek istek:

```
curl -X POST http://localhost:5266/api/search \  
-H "Content-Type: application/json" \  
-d '{"query": "teknik borç nasıl yönetilir?", "topK": 5}'
```

5 Testler ve Deneyler

```
dotnet test # 44 xUnit testi  
dotnet run --project experiments/ColdStart.Experiments -- transition # LLM'siz, ücretsiz  
dotnet run --project experiments/ColdStart.Experiments -- activation --with-llm  
dotnet run --project experiments/ColdStart.Experiments -- sensitivity --with-llm
```

Qdrant container'ı kapalıyken integration testleri otomatik atlanır. `--with-llm` bayraklı deneyler OpenAI çağrısı yapar; sonuçlar `data/results/` altına CSV olarak yazılır.

BELİRTİ	NEDEN / ÇÖZÜM
Layer 2/3'te "OpenAI API anahtarı yapılandırılmamış" hatası	<code>OPENAI_API_KEY</code> environment variable'ını tanımlayıp uygulamayı yeniden başlatın.
Layer 3'te Qdrant bağlantı hatası	<code>docker compose up -d</code> çalıştığından emin olun; <code>docker ps</code> ile container'ı doğrulayın.
<code>Vector dimension error</code> / Qdrant panic	İmaj sürümü değiştirilmiş olabilir; <code>v1.18.1</code> 'e dönüp volume'u sıfırlayın (<code>docker compose down -v</code>).
Port 5266 kullanımda	<code>dotnet run --project src/ColdStart.Api --urls http://localhost:5300</code> gibi farklı bir port verin.